RAG-Powered Music Recommender

Retrieval-Augmented Generation using ChromaDB & Hugging Face LLM — a semantic music recommendation system that understands natural language queries like *"sad songs for a rainy day."*

NLP | RAG | CHROMADB | HUGGING FACE

Bobbala Sakshitha

Project Objectives

What the Project Required

- Build a full RAG pipeline: query → retrieval → prompt → generation
- Use **ChromaDB** for vector storage and **Hugging Face LLM** for generation
- Create a user-friendly **Gradio** interface for conversational QA
- Evaluate retrieval accuracy, response quality, and latency

My implementation: a music recommender that answers natural language mood-based queries using semantic search over 1,000 Spotify songs.

What Is Retrieval-Augmented Generation?

RAG combines the precision of **information retrieval** with the fluency of **generative language models** — producing answers grounded in real, verifiable data.

🔍 Retrieve

Fetch relevant documents from a vector knowledge base using semantic similarity

📝 Augment

Inject retrieved context into the prompt before sending to the LLM

💬 Generate

LLM produces a grounded, accurate, human-readable response

Why RAG? It reduces hallucinations, makes answers verifiable from sources, and allows the knowledge base to be updated without retraining the model.

System Architecture

User Query

Embedding Model

ChromaDB Retrieval

Prompt Engineering

Hugging Face LLM

Each stage is modular — the embedding model, vector store, and LLM can be swapped independently, enabling experimentation with different components without redesigning the full pipeline.

Data Preparation

Dataset: 200K+ Spotify Songs (Kaggle)

1,000 songs were selected and converted into structured text chunks for embedding. Each chunk captures the musical identity of a track:

```
TITLE: Lay My Burden Down
ARTIST: Alison Krauss
GENRE: Country
EMOTION: sadness
ENERGY: 28/100
```

This format gives the embedding model rich, multi-dimensional context — mood, genre, and energy — enabling nuanced semantic matching.

Why This Dataset?

- 🎭 Emotion/mood labels (joy, sadness, anger, etc.)
- 🎵 Real song metadata: artist, genre, year
- ⚡ Audio features: energy, danceability scores
- 🎁 No API needed — CSV download from Kaggle

✅ Output: 1,000 structured text chunks, ready for embedding and ingestion into ChromaDB.

Building the Vector Database

ChromaDB Ingestion Pipeline

```
# build_vector_db.py
model = SentenceTransformer("all-MiniLM-L6-v2")
client = chromadb.PersistentClient(path="./chroma_db")
collection = client.create_collection("songs_collection")

for i, chunk in enumerate(chunks):
    emb = model.encode(chunk,
          normalize_embeddings=True)
    collection.add(
        documents=[chunk],
        embeddings=[emb.tolist()],
        ids=[str(i)]
    )
```

Result: 1,000 song embeddings stored as 384-dimensional vectors, occupying less than 10 MB on disk.

Key Design Decisions

1. **Embedding Model**

   `all-MiniLM-L6-v2` — chosen for its speed/quality balance on CPU hardware

2. **Normalised Embeddings**

   Ensures cosine similarity search is consistent across all vectors

3. **Persistent Storage**

   ChromaDB persists to disk — no re-embedding needed on restart

RAG Pipeline: Retrieval + Generation

```
# rag_engine.py
def search_songs(question, k=3):
    q_emb = model.encode(question,
            normalize_embeddings=True)
    results = collection.query(
        query_embeddings=[q_emb.tolist()],
        n_results=k)
    return results['documents'][0]

def generate_answer(question, context):
    prompt = f"""Based on these songs,
    answer the question.
    Question: {question}
    Songs: {context}
    Recommend songs and explain why."""
    inputs = tokenizer(prompt,
            return_tensors="pt")
    outputs = llm.generate(
            inputs.input_ids,
            max_length=200)
    return tokenizer.decode(outputs[0],
            skip_special_tokens=True)

def ask(question):
    songs = search_songs(question)
    answer = generate_answer(question, songs)
    return answer, songs
```

LLM: FLAN-T5-Small

CPU-Friendly

No GPU required — runs on standard hardware

2–4s Latency

Acceptable response time for a demo system

Open Source

Free via Hugging Face — no API keys needed

The `ask()` function encapsulates the entire RAG pipeline in a single, reusable call — clean separation of retrieval and generation logic.

Gradio Web Interface

Interface Code

```
# app.py
with gr.Blocks(title="Music Recommender") as demo:
  gr.Markdown("# RAG Music Recommender")
  question = gr.Textbox(
      label="What kind of music?",
      placeholder="sad songs for a rainy day")
  k_value = gr.Slider(1, 5, value=3,
      label="Songs to retrieve")
  submit = gr.Button("Get Recommendations")
  answer = gr.Textbox(label="AI Response",
      lines=6)
  songs_display = gr.Textbox(
      label="Retrieved Songs (ChromaDB)",
      lines=8)
  submit.click(fn=process_query,
      inputs=[question, k_value],
      outputs=[answer, songs_display])
demo.launch(share=True)
```

Interface Features

1

Natural Language Input

Free-text query box — no structured commands needed

2

Adjustable Retrieval (k)

Slider lets users control how many songs are retrieved (1–5)

3

Dual Output Panel

Displays both the LLM response and raw retrieved songs side-by-side for transparency

`share=True` generates a public Gradio URL — no server setup required for sharing the demo.

Evaluation & Experiments

Retrieval Accuracy Results

Query	Expected	Actual	Result
"depressing songs"	Sadness mood	Sadness songs	✅ Pass
"happy upbeat songs"	Joy mood	Joy + comedy	⚠️ Partial
"rock music"	Rock genre	Rock genre	✅ Pass

Latency Breakdown

ChromaDB Query	<0.5s
Embedding Generation	~0.3s
LLM Generation	2–4s

Experiments Performed

Embedding Model

`all-MiniLM-L6-v2` selected for best speed/quality trade-off on CPU

LLM Selection

FLAN-T5-small — no GPU required, functional on standard hardware

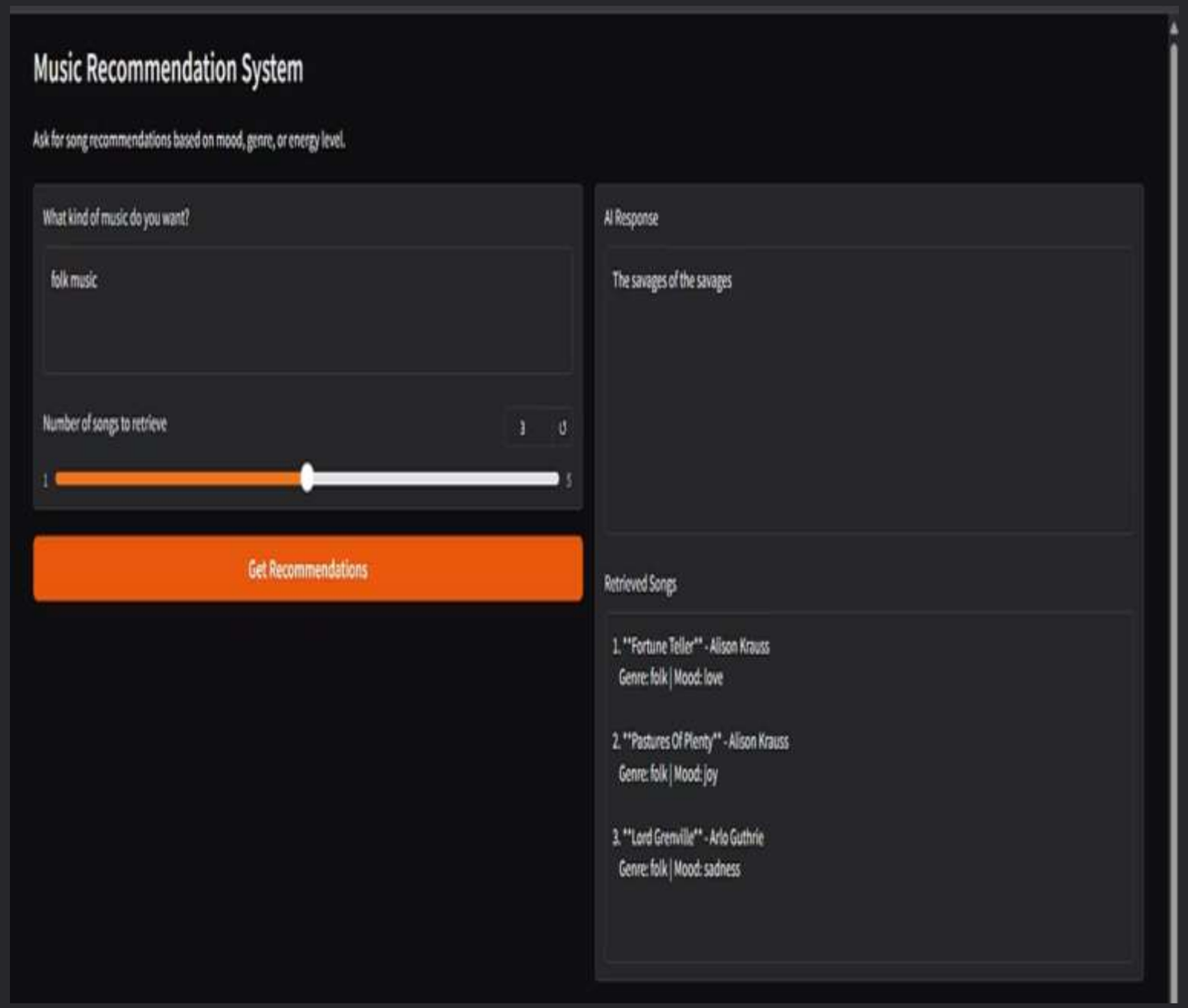
Prompt Templates

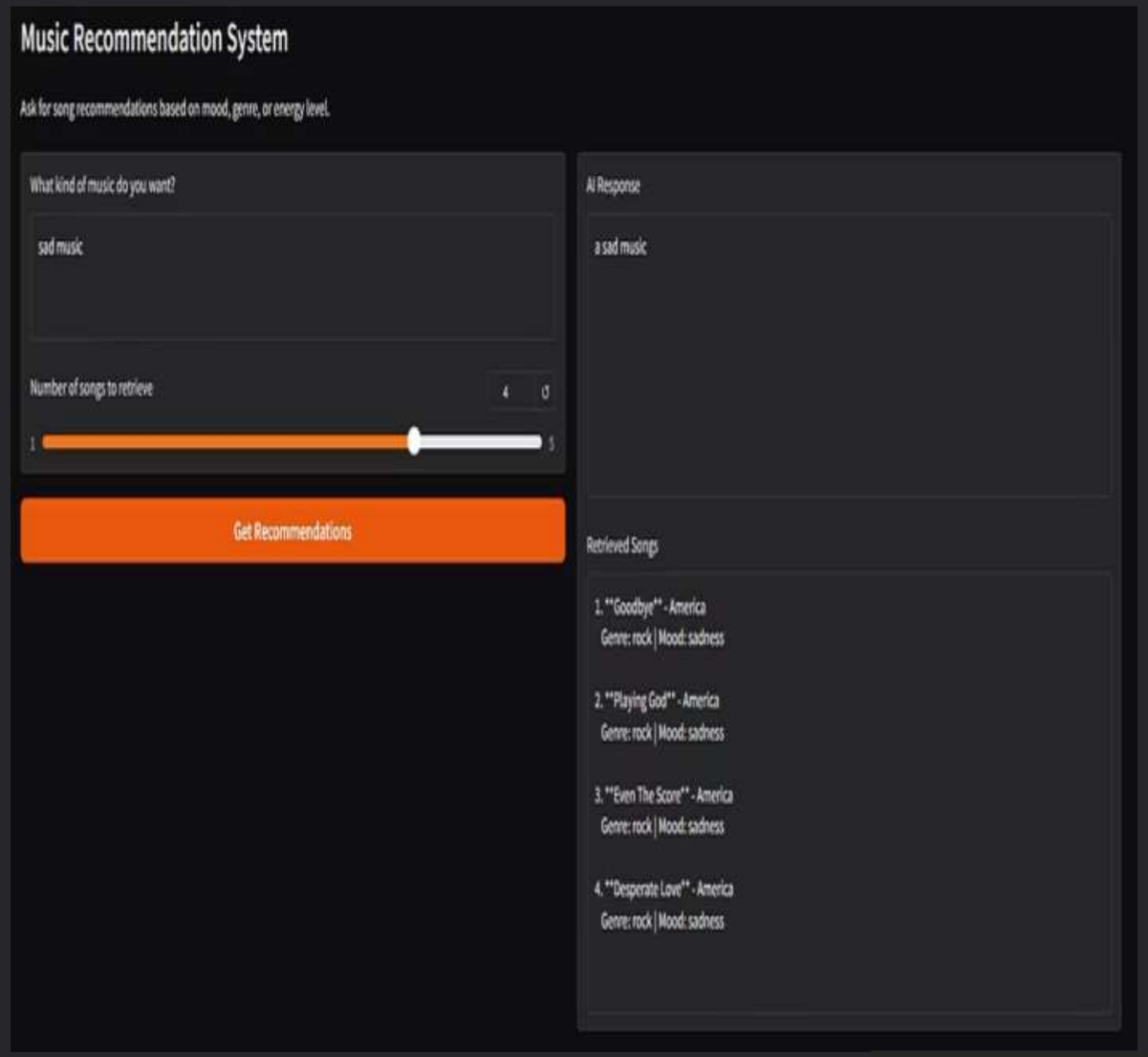
Tested 3 variations; chose the one that produced the most relevant responses

Scalability

ChromaDB supports millions of vectors; current 1,000-song index uses <10 MB

Demo Screenshots





Problems Encountered & Solutions

Problem	Solution
Spotify API 403 Error - Required premium account for audio features	Switched to Kaggle dataset (no API needed)
Hugging Face pipeline error - 'text2text-generation' task not recognized	Used AutoTokenizer + AutoModelForSeq2SeqLM directly
ChromaDB collection name mismatch - Code looked for "songs" but DB had "songs_collection"	Fixed collection name in all files
LLM generated empty responses - FLAN-T5-small sometimes returns blank	Added fallback formatting in code
Windows symlink warning - Hugging Face cache issue on Windows	Ignored warning - no impact on functionality
Mood labels incorrect - "Happy Happy Birthday Song" labeled as sadness	Documented as dataset limitation in evaluation

Conclusions & Key Takeaways

✅ Fully Functional RAG System

End-to-end pipeline from natural language query to grounded LLM response

🔍 Semantic Search Works

ChromaDB + MiniLM enables mood and genre retrieval without keyword matching

⚙️ Small LLMs Are Viable

FLAN-T5-small produces useful results on CPU — no cloud GPU required

📦 Dataset Quality Matters

Mislabelled mood tags directly impact retrieval accuracy — a key lesson learned

Thank you!